

Dear WEBFLEET.connect partners,

Following our previous communications regarding the decommissioning of URL-based login credentials for the WEBFLEET.connect CSV/JSON API, we are providing information to support the migration to HTTP Basic Authentication.

This migration must be completed by **30th of June 2026**, to avoid disruption.

Please ensure this information is shared with the developer or technical team responsible for maintaining your WEBFLEET.connect integration.

Why is this changing?

Support for login credentials passed as URL query parameters is being discontinued for security reasons and based on customer and integrator demand.

When does this happen?

This change takes effect on **30th of June 2026**.

After this date, API requests that include the parameters username and password in the URL will fail and return authentication error code 1180.

Who needs to take action?

This change applies to partners who:

- Use the WEBFLEET.connect CSV/JSON API
- Currently pass username and password as URL query parameters

No action is required if:

- Your integration or tool already uses HTTP Basic Authentication
- You use the SOAP interface only

What needs to be changed?

You must update your integrations or tools to use HTTP Basic Authentication. Instead of sending the username and password as URL parameters, the application should include them in the HTTP Authorization header. The level of effort required will depend on how your integration is currently implemented.

How does HTTP Basic Authentication work?

With HTTP Basic Authentication, the username and password are no longer sent as URL parameters. Instead, they are combined into a single Base64-encoded string and included in the HTTP(S) request header as the authentication value.

How are username and password encoded?

The username and password are concatenated using a colon and then Base64-encoded.
For example: myusername:mypassword → bXl1c2VybmFtZTpteXBhc3N3b3Jk

How does the Authorization header look?

The HTTP request must include an Authorization header. The header value consists of the word Basic, followed by a space, and the Base64-encoded credentials.

Example implementations

The following examples demonstrate how to add the HTTP Authorization header when making API requests. These examples are provided for reference only. Your implementation may differ depending on the programming language, tool, or integration setup you use.

Visual Basic for Applications (VBA): This example shows how to add the HTTP Authorization header to a request.

```
1. Dim Req As WinHttp.WinHttpRequest
2. Req.SetRequestHeader "Authorization", "Basic bXl1c2VybmFtZTpteXBhc3N3b3Jk"
```

Java: This example shows how to make an API request using HTTP Basic Authentication.

```
1. HttpRequest request = HttpRequest.newBuilder()
2.   .uri(URI.create("https://csv.webfleet.com/extern?" +
3. "lang=en&account=***&apikey=***&action=geocodeAddress&outputformat=json&freetext=Leipzig"))
4.   .header("authorization", getBasicAuthenticationHeader("myusername", "mypassword"))
5.   .build();
6.
7. HttpClient client = HttpClient.newBuilder()
8.   .build();
9.
10. HttpResponse<String> response;
11. try {
12.     response = client.send(request, BodyHandlers.ofString());
13.     System.out.println(response.body())
14. } catch (Exception e) {
15.     e.printStackTrace();
16. }
17.
18. private static final String getBasicAuthenticationHeader(String username, String password) {
19.     String valueToEncode = username + ":" + password;
20.     return "Basic " + Base64.getEncoder().encodeToString(valueToEncode.getBytes());
21. }
```

C#: This example shows how to make an API request using HTTP Basic Authentication.

```
1. HttpClient client = new HttpClient();
2. string baseUrl = "https://csv.webfleet.com/extern";
3. Uri baseUri = new Uri(baseUrl);
4. client.BaseAddress = baseUri;
5.
6. var authenticationString = "myusername:mypassword";
7. var base64EncodedAuthenticationString =
8.     Convert.ToBase64String(
9.         System.Text.ASCIIEncoding.ASCII.GetBytes(authenticationString)
10.    );
11.
12. var requestMessage = new HttpRequestMessage(
13.     HttpMethod.Get,
14.     "?lang=en&account=***&apikey=***&action=geocodeAddress" +
15.     "&outputformat=json&freetext=Leipzig"
16. );
17.
18. requestMessage.Headers.Authorization =
19.     new AuthenticationHeaderValue("Basic", base64EncodedAuthenticationString);
20.
21. var response = await client.SendAsync(requestMessage);
```

Support

If you encounter issues or have questions about how HTTP Basic Authentication works in WEBFLEET.connect, please contact us or your account manager. We strongly recommend starting your migration early to avoid any last-minute disruption.

Thank you for your continued collaboration.

Kind regards,
Your Webfleet team